



Name of the module: vector

Location in GIT repository: vspa_common\vspa-lib\vector

Overview:

This module provides kernels for performing the dot product between 2 complex buffers $x_2[n]$ and $x_1[n]$ with the expression:

$$dot_prod = \sum_{n=0}^{L-1} x_2[n]x_1[n]^*$$

Type of code:

- VCPU assembly functions
 - dot_prod_circ_x32chf_csp_asm
 - dot_prod_line_x32chf_csp_asm

Assembly Functions API:

```
void dot_prod_circ_x32chf_csp_asm( vspa2_complex_fixed16* inp1_p,  
                                   vspa2_complex_fixed16* inp2_p,  
                                   vspa_complex_float32* dot_prod_p,  
                                   vspa2_complex_fixed16* inp_circ_p,  
                                   size_t inp_circ_size,  
                                   uint32_t num_lines);
```

where:

inp1_p – Pointer of 1st input buffer (16-bit complex half-fixed, non vector aligned).
inp2_p – Pointer of 2nd input buffer (16-bit complex half-fixed, non vector aligned)
dot_prod_p – Pointer for output dot product (32-bit complex single-precision, non vector aligned).
inp_circ_p – Input circular buffer start address (16-bit complex half-fixed, vector aligned).
inp_circ_size – Input circular buffer size (in half-words).
num_lines – Number of input lines (each line has 32 x complex half-fixed samples).

```
void dot_prod_line_x32chf_csp_asm( vspa2_complex_fixed16* inp1_p,  
                                   vspa2_complex_fixed16* inp2_p,  
                                   vspa_complex_float32* dot_prod_p,  
                                   uint32_t num_lines);
```

where:

inp1_p – Pointer of 1st input buffer (16-bit complex half-fixed, non vector aligned).
inp2_p – Pointer of 2nd input buffer (16-bit complex half-fixed, non vector aligned)
dot_prod_p – Pointer for output dot product (32-bit complex single-precision, non vector aligned).
num_lines – Number of input lines (each line has 32 x complex half-fixed samples).

Implementation plan

The implementation will consist in two phases:

- 1st phase:
 - take first group of 16 (# AU) samples from each input buffer and perform the complex conjugate element-wise multiplication in the destination register $V = x_2[0..15]x_1[0..15]^*$



- take next groups of 16 (# AU) samples from each input buffer and accumulate the complex conjugate element-wise multiplication to the destination register $V = V + x_2[16 * k + 0..15]x_1[16 * k + 0..15]^*$ using the 'cmac' instruction
- 2st phase:
 - The destination register will be $V = x_2[0..15]x_1[0..15]^* + x_2[16..31]x_1[0..31]^* + \dots$ at the end of the 1st phase. The 2nd phase will sum up all the elements of the V register in order to obtain the final dot product.

Memory management

Two different memory allocation patterns are assumed:

- *vector aligned linear allocation*: both input buffers (inp1_p and inp2_p) are DMEM vector-aligned and linearly allocated (linear addressing used). Kernels which assume this allocation use the “_line_” naming:
 - **dot_prod_line_x32chf_csp_asm**
- *vector non-aligned circular allocation*: both input buffers (inp1_p and inp2_p) are NOT required to be DMEM vector-aligned and are circular. Both input buffers use circular addressing inside a common circular buffer described by a start address (inp_circ_p) and a size (inp_circ_size). Kernels which assume this allocation use the “_circ_” naming:
 - **dot_prod_circ_x32chf_csp_asm**

Limitations

The approach described above has the following limitations:

- The dot product window length is limited to a multiple of 32 samples

DMEM requirements

Variable	Minimum size (half-words)	Minimum alignment (half-words)	Allocated by
inp1_p	64	64/none	Caller
inp2_p	64	64/none	Caller
dot_prod_p	4	none	Caller

L-LTF frequency offset estimation:

For frequency offset estimation the time-domain autocorrelation between the 1st and 2nd L-LTF symbol is used. The autocorrelation metric is expressed as:

$$acorr = \sum_{n=0}^{W_{acorr}-1} r[n+64]r[n]^*$$

where $r[n]$ is the IQ input signal of the correlator with $r[0]$ corresponding to 1st sample of the 1st L-LTF symbol and with $r[n+64]$ corresponding to 1st sample of the 2nd L-LTF symbol (assuming correct boundary was previously detected). The sampling rate is assumed 20 MHz when the L-LTF symbol has 64 samples. The dot product above is performed for W_{acorr} samples.

Using the autocorrelation complex metric the frequency offset is estimated as:



$$FO_{est}[Hz] = \frac{\arg\{acorr\} F_s[Hz]}{2\pi \cdot 64}$$

where $\arg\{\cdot\}$ is the phase (in radians) of a complex value and F_s is the sampling rate.

This module provides the function **dot_prod_circ_x32chf_csp_asm** which can be used since:

- a 1-point autocorrelation is actually a dot product between $r[n]$ and $r[n + 64]$
- the input signal of the correlator is expected to be streamed in a circular manner
- the start of the 1st buffer ($r[n]$) is indicated by the L-LTF detection which can be misaligned

Test plan:

- **Tested features**
 - Test different dot product lengths (32, 64, ...)
 - Test different buffer allocations types (vector aligned linear allocation and non-aligned circular allocation)
- Take a look in **Dot_Prod_Testplan.xlsx** for test plan details